

```
const form = document.querySelector('form');
```

Eventos y Validación de Formularios en JavaScript

```
const formEmail(form) {  
  const form.email = "mo";  
});
```

```
event.preventDefault();
```

Defendiendo el Punto de Control del DOM

```
const email = 0;
```

```
if (input.value === '') {
```

```
  (input.value === '')
```

```
  console.log("U");
```

```
  return;
```

```
} else {
```

```
  validateEmail(email);
```

```
}
```

```
}
```

```
}
```

Enviar



3° de Bachillerato en TI – Desarrollo Front End

El Sistema Nervioso del DOM

Un **evento** es cualquier acción que ocurre en la página web. Sin eventos, el Front End es estático; con ellos, la interfaz reacciona.

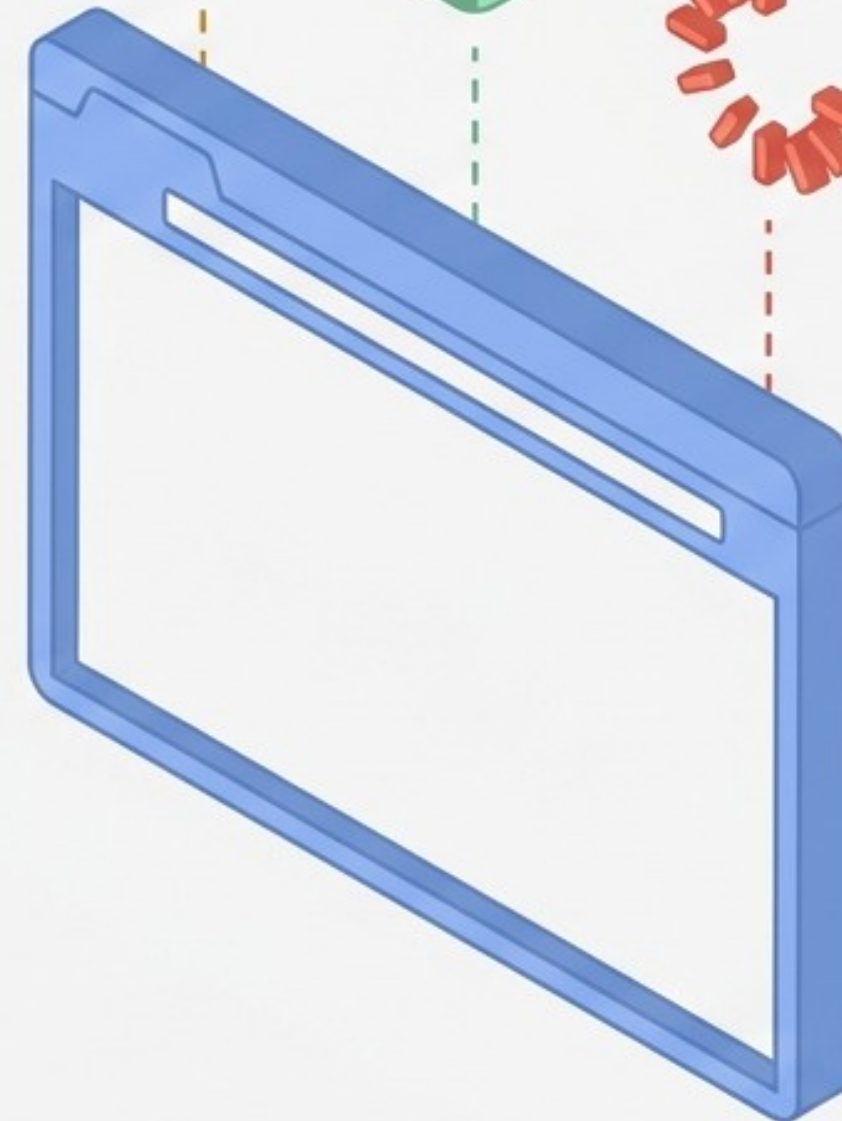
Ratón: click,
mouseenter



Teclado: keyup,
keydown



Ventana:
load



Asignando a nuestro Guardia: Manejadores de Eventos

Arcaico (En línea)

```
<button onclick='alerta()'>
```



Limitado. Sobrescribe otros eventos y mezcla HTML con la lógica de programación.

Estándar Moderno (El Listener)

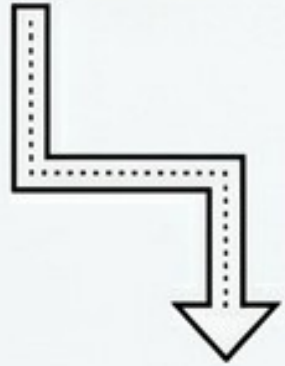
```
elemento.addEventListener()
```



Robusto. Permite múltiples escuchas sobre un mismo elemento sin pisarse entre sí.

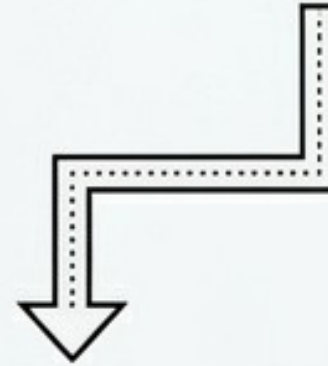
Anatomía de addEventListener

```
elemento.addEventListener('click', validar, false);
```



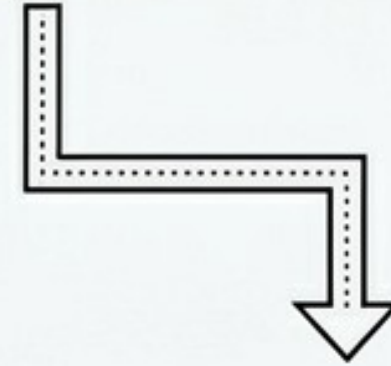
¿Quién escucha?

El nodo del DOM que hemos capturado previamente.



¿Qué estímulo?

El nombre del evento en formato cadena de texto.



¿Qué hace?

La función (manejador) que se ejecuta como respuesta.

Código en Acción: Alterando el DOM

```
const boton = document.querySelector('#miBoton');  
const mensaje = document.querySelector('#mensaje');  
  
boton.addEventListener('click', () => {  
  mensaje.textContent = '¡Acceso Autorizado!';  
  mensaje.style.color = 'green';  
});
```



El Momento Crítico: El Evento submit

Cuando un usuario presiona Enviar, el navegador asume que los datos están listos y **recarga** la página por defecto.



Deteniendo el Tiempo: `preventDefault()`

```
1 formulario.addEventListener('submit', function(event) {  
2   event.preventDefault();  
3   // La recarga de la página ha sido bloqueada.  
4   // Ahora nosotros tenemos el control.  
5 });
```



Inspección Manual: Extrayendo Valores

```
1 const contrasenia = document.querySelector('#pass').value;  
2  
3 if (contrasenia.length < 8) {  
4     // Falla: Contraseña muy corta  
5 } else if (contrasenia === '') {  
6     // Falla: Campo vacío  
7 } else {  
8     // Éxito: Datos correctos  
9 }
```



El Escáner Automático: API de Validación HTML5

En lugar de escribir docenas de condicionales, HTML5 nos ofrece herramientas nativas integradas en el navegador.



Matriz Diagnóstica: El Objeto validity

Error del Usuario (Input)	Propiedad de la API	Estado Booleano
Campo obligatorio vacío	elemento.validity.valueMissing	 true
Escribe "hola" en campo Email	elemento.validity.typeMismatch	 true
No respeta el pattern de seguridad	elemento.validity.patternMismatch	 true
Datos ingresados correctamente	elemento.validity.valid	 true

Tomando el Control: `setCustomValidity()`

```
1 if (elemento.validity.valueMissing) {  
2   elemento.setCustomValidity('¡Alto!  
   Necesitamos tu nombre.');
```



```
4 } else {  
5   elemento.setCustomValidity(''); //  
   Anula el error  
6 }
```

Regla de Oro: Declarar un texto invalida el formulario. Declarar una cadena vacía '' lo valida.

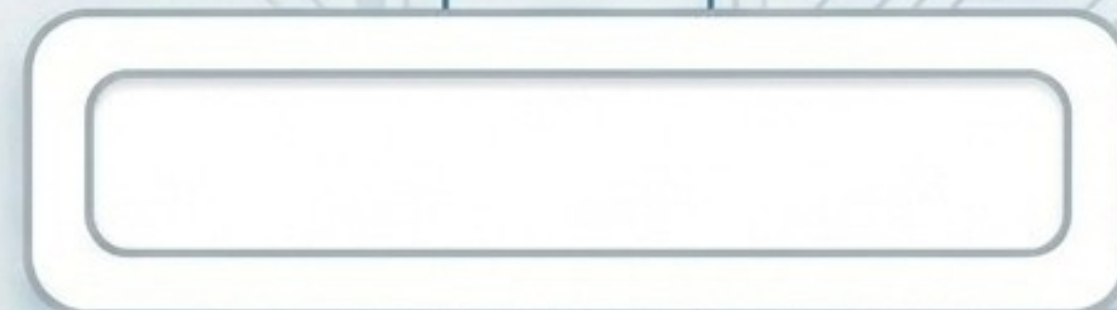
Nombre Completo

¡Alto! Necesitamos tu nombre.

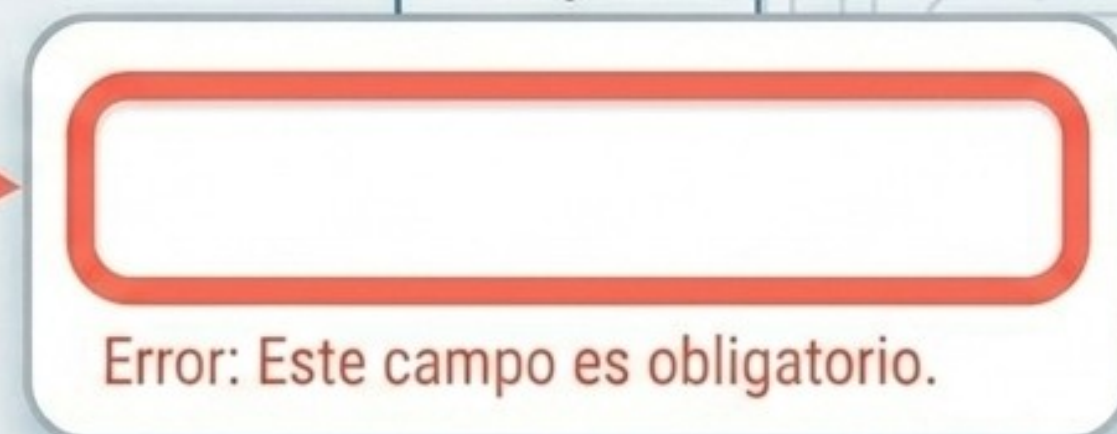
Retroalimentación Visual: UX/UI en el DOM

```
function mostrarError(elemento, mensaje) {  
  elemento.style.borderColor = '#FFDDDD';  
  cajaMensaje.textContent = mensaje;  
}
```

Antes

A white rounded rectangular input field with a thin grey border, representing the state before an error is shown.

Después

The same white rounded rectangular input field, but now with a thick red border. Below the field, the text "Error: Este campo es obligatorio." is displayed in a red font.

Síntesis: El Flujo Completo de Validación



Ejercicio Práctico: Alta de Pacientes

"HR Clínicas"

Misión

Tu misión: Construir el punto de control seguro.

- ☐ Escuchar el evento submit y aplicar `preventDefault()`
- ☐ Validar usando la API de HTML5 (`valueMissing` para Nombre, `patternMismatch` para Cédula)
- ☐ Utilizar `.textContent` y `.style` para mostrar alertas rojas si hay errores
- ☐ Mostrar mensaje de éxito verde si todo es correcto

El Objetivo

HR Clínicas - Nuevo Paciente

Nombre

Cédula

Email

Registrar